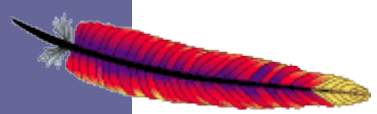
JavaServer Pages 2.0 Technology:
The Community Delivers!

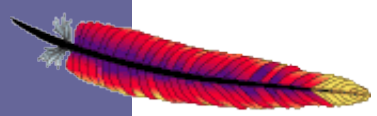
Mark Roth
JSP 2.0 Specification Lead
Sun Microsystems, Inc.

ApacheCon 2003
11-17-2003, Session MO-06, 13:30-14:30

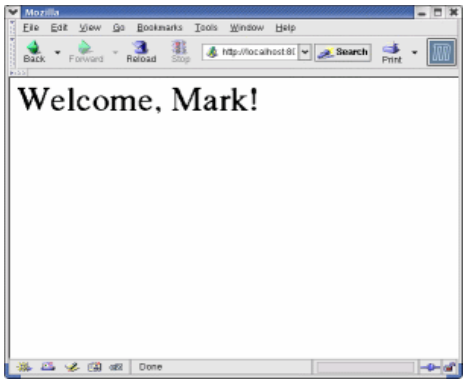


Overview

- JSP technology refresher (by example)
- Motivation for JSP 2.0 technology
- Exploration of new features
  - Integrated expression language
  - Simple tag extensions
  - Tag files
  - Improved XML syntax
  - Other features
- The Role of the Community

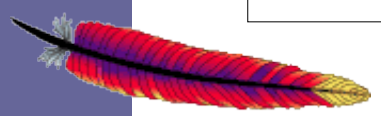


Servlet API Technology

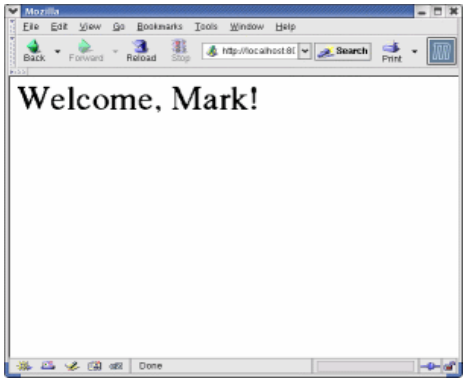




```
protected void doGet( HttpServletRequest req,
                      HttpServletResponse resp )
{
    PrintWriter out = resp.getWriter();
    out.println( "<html><body>" );
    out.println( "Welcome, " +
        req.getParameter( "name" ) + "!" );
    out.println( "</body></html>" );
    ...
}
```

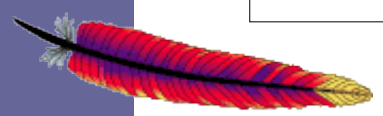


Classic JSP Technology



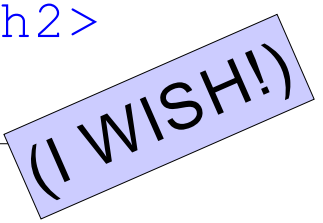


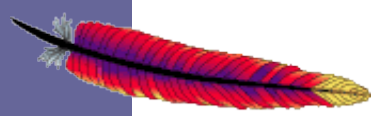
```
<html><body>
Welcome, <%= request.getParameter( "name" ) %>!
</body></html>
```



Classic JSP Technology: Template Text

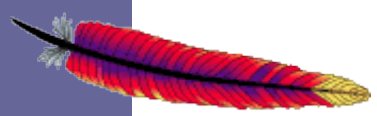
```
<html><body>
  <h2><img src="images/feather.gif" align="baseline">
     JavaServer Pages Demo</h2>
  <hr>
  Symbol SUNW is now 59.20
  <hr>
  <small><i>JSP Demonstration powered by SunONE
  Application Server, build on top of Tomcat
  </i></small>
  <div align="right"><img src="images/tomcat.gif">
  </div>
</body></html>
```





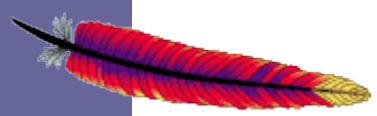
Classic JSP Technology: Directives

```
<html><body>
  <%@ include file="/WEB-INF/jspf/header.jspf" %>
  Symbol SUNW is now 59.20
  <%@ include file="/WEB-INF/jspf/footer.jspf" %>
</body></html>
```



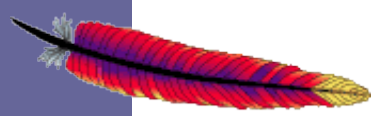
Classic JSP Technology: Scriptlet Expressions

```
<html><body>
  <%@ include file="/WEB-INF/jspf/header.jspf" %>
  Symbol <%= request.getParameter( "symbol" ) %>
  is now 59.20
  <%@ include file="/WEB-INF/jspf/footer.jspf" %>
</body></html>
```



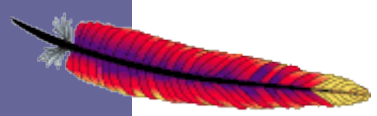
Classic JSP Technology: Scriptlets

```
<% String sym = request.getParameter( "symbol" ); %>
<html><body>
  <%@ include file="/WEB-INF/jspf/header.jspf" %>
    Symbol <%= sym %> is now 59.20
  <%@ include file="/WEB-INF/jspf/footer.jspf" %>
</body></html>
```



Classic JSP Technology: Standard and Custom Actions

```
<%@ taglib prefix="scrp"
           uri="http://jakarta.apache.org/taglibs/«
                scrape-1.0" %>
<% String sym = request.getParameter( "symbol" ); %>
<html><body>
  <%@ include file="/WEB-INF/jspf/header.jspf" %>
  Symbol <%= sym %> is now
  <scrp:page time="20">
    <scrp:url>http://localhost:8080/demo/yahoo/«
              q.jsp?s=<%= sym %></scrp:url>
    <scrp:scrape id="qt" begin="<big><b>" end="</b>"/>
  </scrp:page>
  <jsp:useBean id="qt" class="java.lang.String" />
  <%= qt %>
  <%@ include file="/WEB-INF/jspf/footer.jspf" %>
</body></html>
```



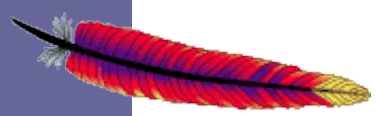
Classic JSP Technology: Miscellaneous

- Declarations:

```
<%!
    public String getSymbol() {
        return "SUNW";
    }
%>
```

- Comments:

```
<%-- JSP Comments --%>
```



Focus of JSP 2.0 Technology

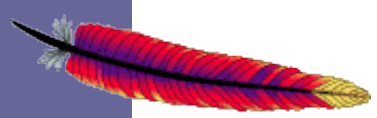
- Ease of use – lowers the bar

JSP 1.2

JSP 2.0

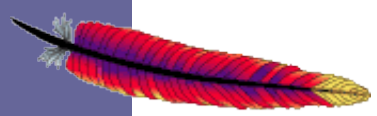
User Class	HTML	XML	Java
Tag Library Developer			
Advanced Page Author			
Basic Page Author			—

 = Basic Knowledge  = Expert



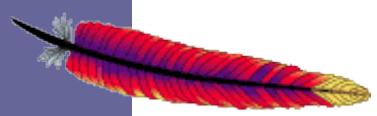
JSP 1.2 Syntax With Scriptlets

```
<%-- Output Shopping Cart --%>
<%@ page import="apachecon2003.*" %>
<%@ taglib prefix="util" uri="http://mytaglib" %>
<html><body>
  <util:getShoppingCart var="cart" />
  <table border="1">
  <%  for( int i = 0; i < cart.size(); i++ ) {
        CartItem item = (CartItem)cart.get(i);
  %>
    <tr>
      <td><img src="images/<%=
                        item.getPicture() %>"></td>
      <td><%= item.getName() %></td>
      <td><%= item.getPrice() %></td>
    </tr>
  <%  } %>
  </table>
</body></html>
```



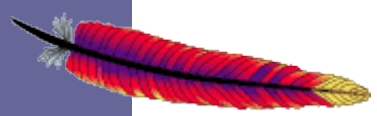
JSP 2.0 Syntax Without Scriptlets

```
<%-- Output Shopping Cart --%>
<%@ taglib prefix="util" uri="http://mytaglib" %>
<%@ taglib prefix="c"
           uri="http://java.sun.com/jsp/jstl/core" %>
<html><body>
  <util:getShoppingCart var="cart" />
  <table border="1">
    <c:forEach var="item" items="${cart}">
      <tr>
        <td><img src="images/${item.picture}"></td>
        <td>${item.name}</td>
        <td>${item.price}</td>
      </tr>
    </c:forEach>
  </table>
</body></html>
```



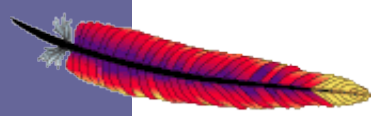
JSP 2.0 Syntax
Improved XML Syntax

```
<!-- Output Shopping Cart -->
<html xmlns:util="http://mytaglib"
      xmlns:c="http://java.sun.com/jsp/jstl/core"
      xmlns:jsp="http://java.sun.com/JSP/Page">
  <jsp:directive.page contentType="text/html" />
  <body>
    <util:getShoppingCart var="cart" />
    <table border="1">
      <c:forEach var="item" items="${cart}">
        <tr>
          <td><img src="images/${item.picture}"/></td>
          <td>${item.name}</td>
          <td>${item.price}</td>
        </tr>
      </c:forEach>
    </table>
  </body>
</html>
```



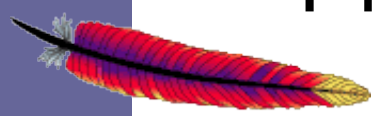
Exploration of New JSP 2.0 Features

- Integrated expression language
- Simple tag extensions
- Tag files
- Improved XML syntax
- Other features



Integrated Expression Language

- Based on "SPEL" from JSTL 1.0
- Example: `${item.price}`
- Recognized by JSP container in:
  - Template text
  - Attributes of any standard or custom action
- Support for custom EL functions:
  - Extensible via tag libraries (JSTL provides 16)
  - Example: `${fn:length(lastName)}`
- Programmatic access



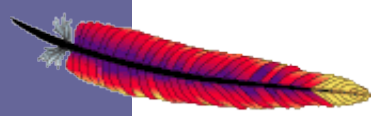
Integrated Expression Language Example

- Using scriptlets:

```
<center>
  <jsp:useBean id="foo" class="FooBean" />
  <%= foo.getBar() %>
</center>
```

- Equivalent, using an EL expression:

```
<center>
  ${foo.bar}
</center>
```



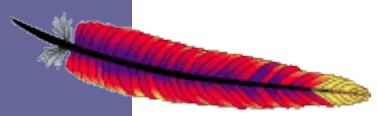
Integrated Expression Language Example

- Using scriptlets:

```
<%   Map m = (Map)pageContext.getAttribute(
          "state" );
     State s = ((State)m.get( "NY" ));
     if( s != null ) {
%>
          <%= s.getCapital() %>
<%   } %>
```

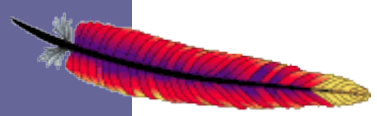
- Equivalent, using an EL expression:

```
${state["NY"].capital}
```



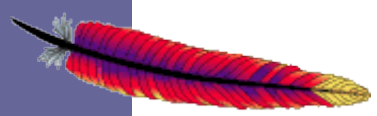
Expression Language Details

- Implicit Objects: `${header['User-Agent']}`
  - pageContext
  - pageScope, requestScope, sessionScope, applicationScope
  - param, paramValues
  - header, headerValues
  - cookie
  - initParam



Expression Language Details

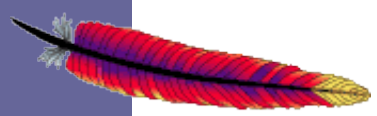
- Variable resolution: `${foo}`
  - Looks in page, request, session, application scopes
- Gracefully handles null values
- Operators `[]` and `.`
  - `${map.key}` or `${map['key']}`
  - `${list[index]}`
  - `${array[index]}`
  - `${bean.property}` or `${bean['property']}`



Expression Language: Other Operators

- In Order Of Precedence:

```
[ ] .
( )
- (unary), not, !, empty
*, /, div, %, mod
+, - (binary)
<, >, <=, >=, lt, gt, le, ge
==, !=, eq, ne
&&, and
||, or
A ? B : C
```



Writing an EL Function

1. Write a static method

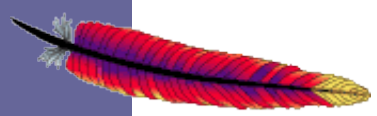
```
public static String titlecase( String s ) {
    return Character.toUpperCase( s.charAt(0) ) +
           s.substring( 1 );
}
```

2. Add `<function>` to the Tag Library Descriptor

```
<function>
  <name>titlecase</name>
  <function-class>com.acme.Util</function-class>
  <function-signature>
      java.lang.String titlecase(java.lang.String)
  </function-signature>
</function>
```

3. Import tag library and use!

```
${fn:titlecase(book.title)}
```

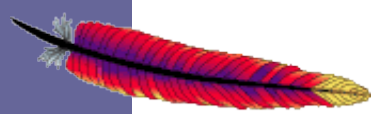


Programmatic Access to EL

- Designed to be used for non-JSP sources
  - Examples: configuration files, user input
- JspContext can return an `ExpressionEvaluator`

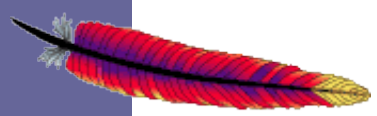
```
public Object evaluate(
        String expression,
        Class expectedType,
        VariableResolver vResolver,
        FunctionMapper fMapper ) throws ELException;

public Expression parseExpression(
        String expression,
        Class expectedType,
        FunctionMapper fMapper ) throws ELException;
```



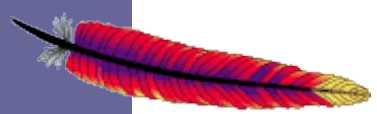
Exploration of New JSP 2.0 Features

- Integrated expression language
- Simple tag extensions
- Tag files
- Improved XML syntax
- Other features



Comparison of Tag Extensions

- Classic tag extensions (JSP 1.2)
  - Inverted tag handler API
  - Written only in the Java programming language
  - Created only by tag library developers
- Simple tag extensions (JSP 2.0)
  - Simpler tag handler API
  - Written in the Java programming language or using JSP syntax
  - Created by page authors or tag library developers



Repeat Tag Implemented as a Classic JSP 1.2 Tag Extension

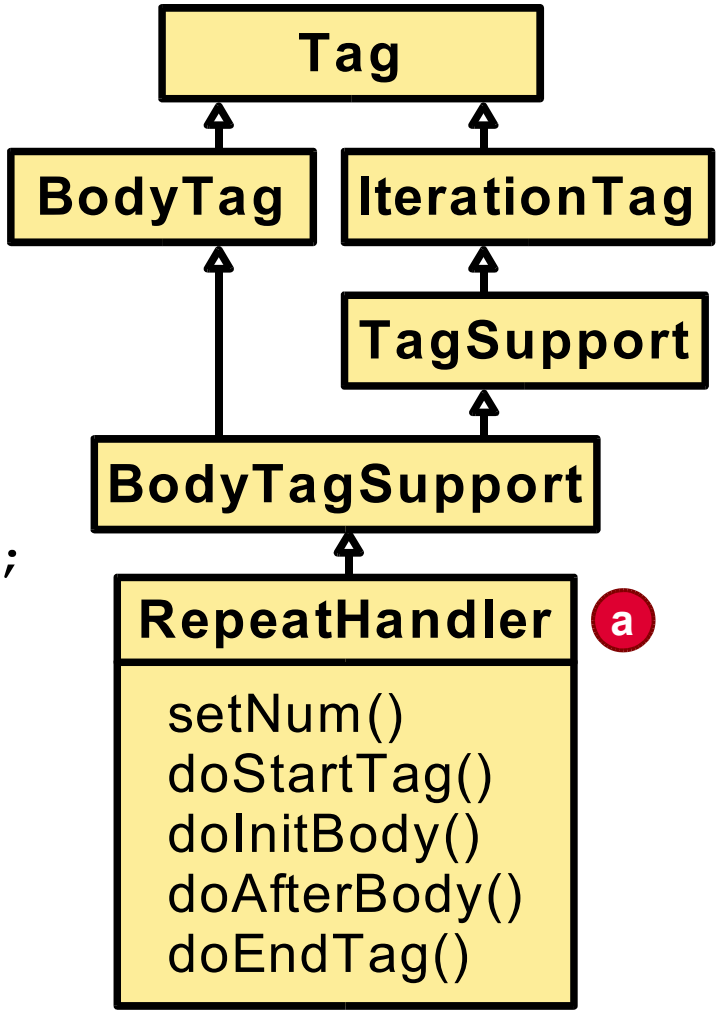
Usage

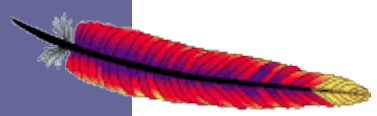
```
<%@ taglib prefix="my"
           uri="/mytags" %>
<my:repeat num="3">
    tag body
</my:repeat>
```

Implementation

```
int doStartTag() {
    this.count = this.num;   (b)
    return Tag.EVAL_BODY_INCLUDE;
}

int doAfterBody() {
    this.count--;   (c)
    return (this.count > 0) ?
      Tag.EVAL_BODY_AGAIN :   (d)
      Tag.SKIP_BODY;
}
```

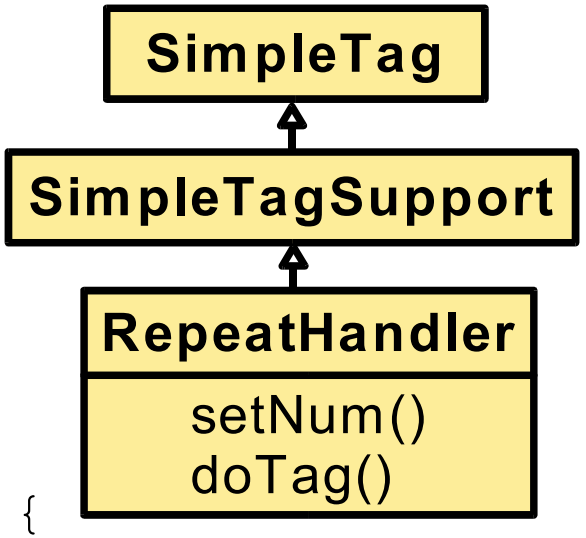




Repeat Tag Implemented as a Simple JSP 2.0 Tag Extension

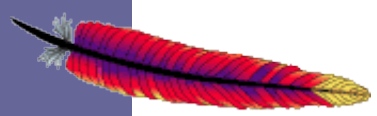
Usage

```
<%@ taglib prefix="my"
           uri="/mytags" %>
<my:repeat num="3">
    tag body
</my:repeat>
```



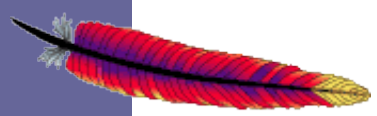
Implementation

```
void doTag() {
    for( int i = 0; i < num; i++ ) {
        getJspBody().invoke( null );
    }
}
```



Exploration of New JSP 2.0 Features

- Integrated expression language
- Simple tag extensions
- Tag files
- Improved XML syntax
- Other features



Tag Files

- Quickly write tag extensions in JSP syntax!
- JSP : Servlet :: Tag File : Tag Handler
- Empowers page authors
- Faster round-trip development
- Simple yet flexible packaging
  - Just drop a .tag file in /WEB-INF/tags/
  - Implicit tag library automatically generated
  - Or, write a .tld for added flexibility
  - Or, package in a JAR with a .tld



Repeat Tag Implemented as a Tag File

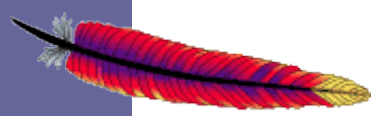
Usage

```
<%@ taglib prefix="my"
           tagdir="/WEB-INF/tags/" %>
<my:repeat num="3">
    tag body
</my:repeat>
```

Implementation

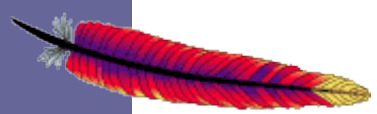
```
<%-- /WEB-INF/tags/repeat.tag --%>
<%@ attribute name="num" %>
<%@ taglib prefix="c"
       uri="http://java.sun.com/jsp/jstl/core" %>

<c:forEach begin="1" end="${num}">
    <jsp:doBody />
</c:forEach>
```



Exploration of New JSP 2.0 Features

- Integrated expression language
- Simple tag extensions
- Tag files
- Improved XML syntax
- Other features

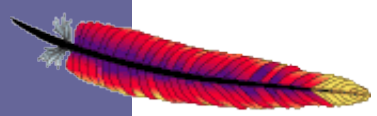


Improved XML Syntax: JSPX and TAGX

- Finally, no more need for `<jsp:root>`!
- JSP technology as an XML namespace
- Ideal for outputting XML content like SVG:

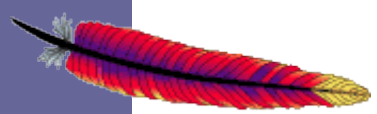
```
<!-- images/hello.svg -->
<svg xmlns:c="http://java.sun.com/jsp/jstl/core"
     width="8.5in" height="11.0in">
  <c:forEach var="y" begin="3" end="10">
    <text style="fill:blue;" y="${y*15}">
      "Hello, ${param.name}!"
    </text>
  </c:forEach>
</svg>
```

- JSP documents (.jspx) or Tag files (.tagx)



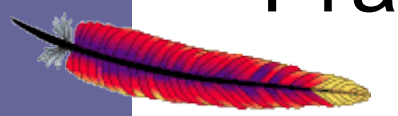
Exploration of New JSP 2.0 Features

- Integrated expression language
- Simple tag extensions
- Tag files
- Improved XML syntax
- Other features



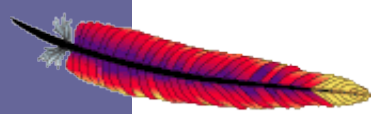
Other Features...

- Central configuration via url-patterns
  - Map extensions other than .jsp
  - Enable / disable scriptlets and EL globally
  - Control page encoding globally
  - Preludes / codas
- Portable debugging support via JSR-45
- Dynamic attributes
- Enhanced I18N support
- Fragment attributes



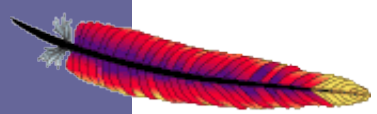
JSP Standard Tag Library (JSTL) Brief Overview

- Integral part of writing scriptless pages
- Use JSTL 1.0 with JSP 1.2
- Use JSTL 1.1 with JSP 2.0:
- Core Tag Library (c:)
  - General-Purpose Actions
  - Conditional Actions
  - Iterator Actions
  - URL-Related Actions



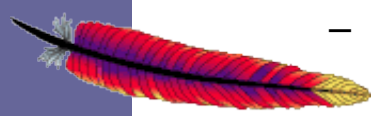
JSP Standard Tag Library (JSTL) Brief Overview

- Formatting Library (fmt:)
  - Internationalization (I18N)
  - Formatting dates, times, numbers
- SQL Actions (sql:)
  - Simple DB queries, updates, transactions
- XML Actions (x:)
  - XPath processing
  - Flow control
  - XSLT Transformations



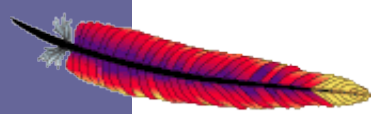
JSP Standard Tag Library (JSTL) Brief Overview

- Tag Library Validators
- Function Library (fn:)
  - `fn:length()`
  - `fn:toLowerCase()`, `fn:toUpperCase()`
  - `fn:substring()`, `fn:substringBefore()`, `fn:substringAfter()`
  - `fn:trim()`
  - `fn:replace()`
  - `fn:indexOf()`, `fn:endsWith()`, `fn:startsWith()`
  - `fn:contains()`, `fn:containsIgnoreCase()`
  - `fn:join()`, `fn:split()`
  - `fn:escapeXml()`



Communities and Standards

- Under development in JCP[SM]: JSR-152
  - Leads: Mark Roth, Eduardo Pelegrí-Llopart
  - Deliverables: Spec, RI, TCK
  - Part of J2EE 1.4 series of specifications
- Very active (and large) expert group
  - Web developers, book authors, tool vendors, container vendors, small and large players.
- Feedback from other specs:
  - JSTL, JavaServer Faces, Servlet, etc.

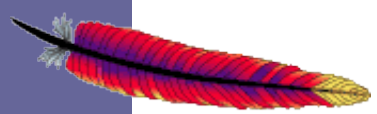


Summary

- JSP 2.0 technology is fully backwards compatible with JSP 1.2 technology
- Focus of JSP 2.0 is ease of use
- Input from the community is key!
- Send your comments to:

  jsp-spec-comments@eng.sun.com
- Get the latest here:

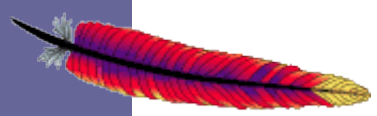
  http://java.sun.com/jsp/



Q&A

Mark Roth
JSP 2.0 Specification Lead
Sun Microsystems, Inc.

mark.roth@sun.com



This presenation has ended.
Click anywhere to return.

_

